COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S4
DATE PERFORMED:	07/24/2026	**DATE SUBMITTED:**	07/24/2026
NAME:	KEITH NICOLE SILVOZA	**INSTRUCTOR:**	JIMLORD QUEJADO

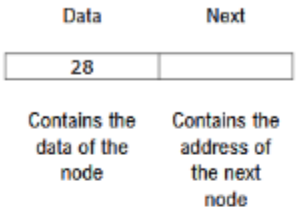
ACTIVITY NO. 3 LINKED LISTS

1. PROCEDURE OUTPUT

ILO A: Construct C++ code for a singly and doubly linked list in C++

A.1. Singly Linked List

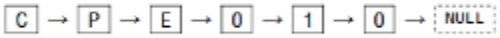
To start, we will do a simple implementation of a linked list. We must keep in mind the visual representation of the individual nodes that will make up our linked list:



Every node in a singly linked list will have 2 compartments, the data and the pointer to the next. The data contains the element of a single type that we want it to contain. The link will point to the next node in our list. In C++, the node is defined as:

```
class Node{
public:
    char data;
    Node *next;
};
```

We will implement a list to represent the string "CPE010" that looks like the figure below:



Implementation will follow the given steps:

1. Create the node pointers and initialize as NULL
2. Create new instances of the node class and allocate them in the heap
3. Define every node in the list
4. Point the last node to null

Simple implementation:

```
#include<iostream>
#include<utility>

class Node{
public:
    char data;
    Node *next;
};

int main(){
    //step 1
    Node *head = NULL;
```

```
        Node *second = NULL;
        Node *third = NULL;
        Node *fourth = NULL;
        Node *fifth = NULL;
        Node *last = NULL;

        //step 2
        head = new Node;
        second = new Node;
        third = new Node;
        fourth = new Node;
        fifth = new Node;
        last = new Node;

        //step 3
        head->data = 'C';
        head->next = second;

        second->data = 'P';
        second->next = third;

        third->data = 'E';
        third->next = fourth;

        fourth->data = '0';
        fourth->next = fifth;

        fifth->data = '1';
        fifth->next = last;

        //step 4
        last->data = '0';
        last->next = nullptr;
    }
```

Although we have created the linked list, this is an implementation that is useless and meant only to show what we want to happen for the given output. **Run your code and screenshot the output, then briefly discuss how the output came to be and how could it be improved (table 3-1)?**

Imagine if we had to make multiple items in the list in the hundreds or thousands! This would be too tedious, ineffective, and inefficient. So, we must implement certain methods. These methods/operations associated with the linked lists are:

- Traversal
- Insertion at head
- Insertion at any part of the list
- Insertion at the end
- Deletion of a node

Linked List Traversal

```
Algorithm: ListTraversal (parameter: pointer to node n)
WHILE n IS NOT EQUAL TO null
      PRINT data OF n
      GO TO NEXT NODE n := next
      ENDWHILE
```

```
    PRINT next line
    END
```

Sometimes we have a linked list, and we need to insert a node somewhere other than at the end of the list. We will look at a couple of different ways to insert a node into an existing list.

To insert a node at the head:

1. Allocate memory for the new node
2. Put our data into the new node
3. Set Next of the new node to point to the previous Head
4. Reset Head to point to the new node

To insert a node at any location between the head and tail:

1. Check if it is the head node (previous node is null)
2. If null, print "Previous node cannot be null."
3. Allocate a new node
4. Store data in the new node
5. Point new node to the node previous node was pointing to
6. Point previous node to the new node

To insert a node at the end:

1. Allocate new node
2. Dereference to the head node
3. Store data in new node
4. Point next of new node to NULL
5. Traverse the list until next of the node is null
6. Point the next of the current node to the new node

To delete a node from linked list:

1. Find previous node of the node to be deleted.
2. Change the next of previous node.
3. Free memory for the node to be deleted.

Create code for each of the pseudocode given for all list operations above.
Provide screenshots in table 3-2.

In your driver function, show the use of each list operation and show the output in table 3-3 found in section 6 with a descriptive caption for each. The tasks you have to perform are as follows:

a. Traverse the list by passing the head of the created list into the function
b. Insert the element 'G' at the start of the list to replace the current node. Output should now show "GCPE101"
c. Insert an element "E" with the previous node element being "P". Output should now show "GCPEE101".
d. Delete the node containing the element C.

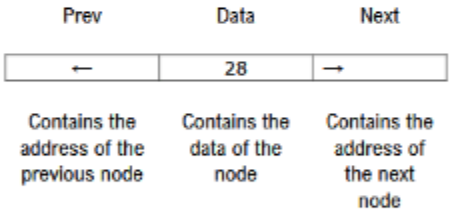
e. Delete the node containing the element P.
f. Show the elements in the list. Output should be "GEE101".

A.2. Doubly Linked List

The singly linked list allows for direct access from a list node only to the next node in the list. A doubly linked list allows convenient access from a list node to the next node and also to the preceding node on the list.



The doubly linked list node accomplishes this in the obvious way by storing two pointers: one to the node following it (as in the singly linked list), and a second pointer to the node preceding it.



This means that in addition to our implementation of the singly linked list, we'll have addition compartment.

```
class Node{
public:
    char data;
    Node *next;
    Node *prev;
};
```

Modify the given operations used in the singly linked lists to work on the new construct of a doubly linked list. Provide a screenshot of your code and analysis in table 3-4.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

SCREENSHOT

```
PS C:\Users\TIPQC\OneDrive\Documents\CPE21S4 - SILVOZA>  & 'c:\Users\TIPQC\.vscode\extensions\ms-vscode.cpptools-1.32.2-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-zinxhzk4.ii3' '--stdout=Microsoft-MIEngine-Out-4dp2phrc.hcg' '--stderr=Microsoft-MIEngine-Error-aa5hhrho.b3f' '--pid=Microsoft-MIEngine-Pid-og2ihkbq.y1t' '--dbgExe=C:\msys64\ucrt64\bin\gdb.exe' '--interpreter=mi'
Original List: List: C P E 0 1 0

Insert G at Head
List: G C P E 0 1 0

Insert E after P
List: G C P E E 0 1 0

Insert X at End
List: G C P E E 0 1 0 X

Delete C
List: G P E E 0 1 0 X

Delete P
List: G E E 0 1 0 X
PS C:\Users\TIPQC\OneDrive\Documents\CPE21S4 - SILVOZA>
```

DISCUSSION

3. Discussion

PART A: What is a linked list?

Linked Lists are a linear data structure. Unlike arrays, linked list elements are not stored at a contiguous location; the elements are linked using pointers.

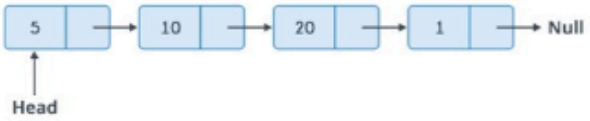


Image Source: Hackearth.com

Why Linked List?

- Arrays are useful but have the following limitations:
- Fixed size.
- Allocated memory remains to be the upper limit.
- Expensive operations (such as insertion) which requires movement of all existing elements and creation of room for new elements.

Advantages over Arrays

- Dynamic size
- Ease of insertion/deletion

Drawbacks

- Random access is not allowed. We have to access elements sequentially starting from the first node
- Extra memory space for a pointer is required with each element of the list.
- Not cache friendly. No locality reference.

Representation:

- A linked list is represented by a pointer to the first node of the linked list. The first node is called the head. If the linked list is empty, then the value of the head is NULL.
- Each node in a list consists of at least two parts:
  - Data
  - Pointer (Or Reference) to the next node

PART B: Doubly Linked Lists

Doubly Linked Lists are traversed in either direction. It is a linked list in which every node has a next pointer and a backpointer.

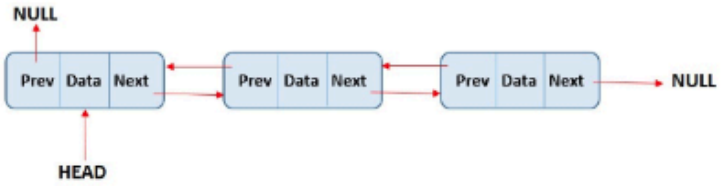


Image Source: AlphaCodingSkills

Every node contains address of next node (except the last node). Every node contains address of previous node (except the first node).

PART C: Common Operations on Linked Lists

Typical operations:

- Initialize the list
- Destroy the list
- Determine if list empty
- Search list for a given item
- Insert an item
- Delete an item, and so on

Operation	

Traversal	
Insertion at head	
Insertion at any part of the list	

```
30    // Traverse the list
31    template <typename T>
32    void dllTraverse(DoubleList<T>* head)
33    {
34    if (head == nullptr)
35    {
36    cout << "The list is empty." << endl;
37    return;
38    }
39
40    DoubleList<T>* current = head;
41    cout << "List: ";
42    while (current != nullptr)
43    {
44    cout << current->data << " ";
45    current = current->next;
46    }
47    cout << endl;
48    }
```

```
50    // Insert at Head
51    template <typename T>
52    void dllInsertHead(T value, DoubleList<T>*& head)
53    {
54    DoubleList<T>* newNode = createNode(value);
55    newNode->next = head;
56    if (head != nullptr)
57    head->prev = newNode;
58    head = newNode;
59    }
```

Insertion at the end

```
61    // Insert at End
62    template <typename T>
63    void dllInsertEnd(T value, DoubleList<T>*& head)
64    {
65    DoubleList<T>* newNode = createNode(value);
66    if (head == nullptr)
67    {
68    head = newNode;
69    return;
70    }
71    DoubleList<T>* current = head;
72    while (current->next != nullptr)
73    current = current->next;
74    current->next = newNode;
75    newNode->prev = current;
76    }
```

Deletion of a node

```
96    // Delete a Node
97    template <typename T>
98    void dllDelete(T value, DoubleList<T>*& head)
99    {
100   if (head == nullptr)
101   return;
102   DoubleList<T>* current = head;
103   while (current != nullptr && current->data != value)
104   current = current->next;
105   if (current == nullptr)
106   return;
107   if (current == head)
108   head = current->next;
109   if (current->next != nullptr)
110   current->next->prev = current->prev;
111   if (current->prev != nullptr)
112   current->prev->next = current->next;
113   delete current;
114   }
```

Source Code

```
// Doubly Linked List Node
template <typename T>
class DoubleList
{
```

Console

```
// Doubly Linked List Node
template <typename T>
class DoubleList
{
public:
T data;
DoubleList<T>* next;
DoubleList<T>* prev;
};
```

Source Code

```
// Create a new node
template <typename T>
DoubleList<T>* createNode(T newData)
{
```

Console

```
// Create a new node
template <typename T>
DoubleList<T>* createNode(T newData)
{
DoubleList<T>* newNode = new DoubleList<T>;

newNode->data = newData;
newNode->next = nullptr;
newNode->prev = nullptr;

return newNode;
}
```

Source Code

```
// Traverse the list
template <typename T>
void dllTraverse(DoubleList<T>* head)
{
```

Console

```
// Traverse the list
template <typename T>
void dllTraverse(DoubleList<T>* head)
{
if (head == nullptr)
{
cout << "The list is empty." << endl;
return;
}

DoubleList<T>* current = head;
cout << "List: ";
while (current != nullptr)
{
cout << current->data << " ";
current = current->next;
}
cout << endl;
}
```

Source Code

```
// Insert at Head
template <typename T>
void dllInsertHead(T value, DoubleList<T>*& head)
{
```

Console

```
// Insert at Head
template <typename T>
void dllInsertHead(T value, DoubleList<T>*& head)
{
DoubleList<T>* newNode = createNode(value);
newNode->next = head;
if (head != nullptr)
head->prev = newNode;
head = newNode;
}
```

Source Code

```
// Insert at End
template <typename T>
void dllInsertEnd(T value, DoubleList<T>*& head)
{
```

Console

```
// Insert at End
template <typename T>
void dllInsertEnd(T value, DoubleList<T>*& head)
{
DoubleList<T>* newNode = createNode(value);
if (head == nullptr)
{
head = newNode;
return;
}
DoubleList<T>* current = head;
while (current->next != nullptr)
current = current->next;
current->next = newNode;
newNode->prev = current;
}
```

Source Code	`// Delete a Node`
	`template <typename T>`
	`void dllDelete(T value, DoubleList<T>*& head)`
	`{`

Console	`// Delete a Node`
	`template <typename T>`
	`void dllDelete(T value, DoubleList<T>*& head)`
	`{`
	`if (head == nullptr)`
	`return;`
	`DoubleList<T>* current = head;`
	`while (current != nullptr && current->data != value)`
	`current = current->next;`
	`if (current == nullptr)`
	`return;`
	`if (current == head)`
	`head = current->next;`
	`if (current->next != nullptr)`
	`current->next->prev = current->prev;`
	`if (current->prev != nullptr)`
	`current->prev->next = current->next;`
	`delete current;`
	`}`
	
	`#endif`

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

COMPUTER ENGINEERING

QUEZON CITY

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.